



Python modules & Packages

By PrudviKrishna





Library & Package

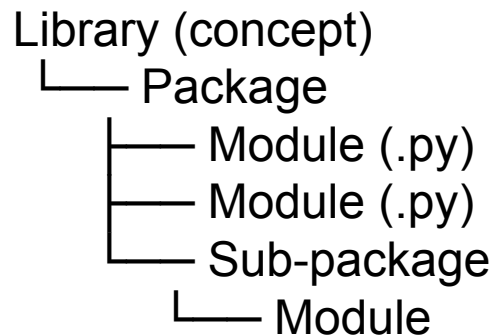
In Python, a **package** is a way of organizing related modules into a single directory hierarchy.

A **library** is a collection of code, and in Python it is usually implemented as one or more packages.

A package is essentially a directory that contains one or more modules (Python files) and

A special file named `__init__.py` and it indicates to Python that the directory should be treated as a package.

Sub-packages: Packages can contain sub-packages, which are just directories within the package that also contain an `__init__.py` file.



```
my_package/  
  __init__.py  
  module1.py  
  module2.py  
  sub_package/  
    __init__.py  
    module3.py
```

```
from my_package import module1  
from my_package.sub_package import module3
```



Modules

A module is a file containing Python code that can define functions, classes, and variables.

Types of Modules:

Standard Library Modules: Built-in modules that come with Python (e.g., math, sys, os).

Third-party Modules: External modules available through package managers like pip (e.g., requests, numpy).

Custom Modules: Modules created by the user to encapsulate functionality.

use “import” keyword to use the modules. **Syntax:** import module_name

Creation and usage of custom module:

Write all the required functionality in .py file in the same folder in main file, Now, in main.py, you can **import** the module and use the defined functions from modulename.py

Ex:

```
import modulename
```

```
Modulename.functionname()
```

Note: please observe the creation of `__pycache__` folder, in the program directory

Namespace

Namespace: It is used to prevent naming conflicts between modules

example, two modules with the same name can exist in different packages without clashing

Syntax: from package_name import module_name

from package_name.module_name import item_name

```
from my_package.sub_package import module3 as sub_module3
from another_package.sub_package import module3 as another_module3

result1 = sub_module3.some_function() # Calls the function from my_package
result2 = another_module3.some_function() # Calls the function from another_package

print(result1) # Output: Result from my_package.sub_package.module3
print(result2) # Output: Result from another_package.sub_package.module3
```

Import

- **Not only modules** — you can import **ANY named object** inside a module:
 - ✓ Module
 - ✓ Class
 - ✓ Function
 - ✓ Variable / Constant
- **Import a specific item (class / function / variable)**
 - `from package_name.module_name import item_name`
- **Import multiple items**
 - `from package_name.module_name import ClassA, function_b, VARIABLE_C`
- **Import everything**
 - `from package_name.module_name import *`

Example:

<code>PI = 3.14</code>	<code>from math_ops import calc</code>	<code># module</code>
	<code>from math_ops.calc import Calculator</code>	<code># class</code>
<code>class Calculator:</code>	<code>from math_ops.calc import add</code>	<code># function</code>
<code> pass</code>	<code>from math_ops.calc import PI</code>	<code># variable</code>
 <code>def add(a, b):</code>		
<code> return a + b</code>		



Thank you